A MINI PROJECT REPORT

on

MOVIE TICKET BOOKING SYSTEM

Subject: Database Management System (DBMS)

Implemented using MySQL

Submitted By	Sanjana Sovanshi
UID	25BCD10108
Submitted To	Mrs. Harkamal Kaur
Designation	Assistant Professor
Submission Date	25 October 2026

Chandigarh University
Gharuan, Mohali, Punjab

Acknowledgement

I would like to express my sincere gratitude to my teacher, Mrs. Harkamal Kaur, Assistant Professor, for her valuable guidance, encouragement and constant support throughout this mini project. Her instructions on how to plan, document and present a database project helped me understand the subject in a practical way.

I am also thankful to Chandigarh University for providing the learning environment and the laboratory facilities needed to complete this work, and to my friends and family for their motivation.

Sanjana Sovanshi
UID: 25BCD10108

Table of Contents

1. Introduction....3
2. Objectives / Problem Statement....4
3. Modules, Tools and Technologies Used....5
4. Database Design (ER Diagram)....6
5. Code with Output....7
5.1 Database and Master Tables....7
5.2 Transaction Tables....8
5.3 Inserting Data....9
5.4 Viewing Table Data....11
5.5 SQL Queries (Joins, Grouping, Aggregates)....12
5.6 View, Transaction and Delete....14
6. Conclusion, Future Scope and References....15

1. Introduction

1.1 Database Management System

A Database Management System (DBMS) is software that allows users to create, store, organise, retrieve and protect data in an efficient and secure way. Instead of keeping information in separate paper registers or scattered files, a DBMS keeps it in structured tables that are linked to each other through keys. This reduces duplication of data, keeps the data consistent, and allows many users to access the same information at the same time. Relational DBMS products such as MySQL, Oracle and PostgreSQL use Structured Query Language (SQL) to define the structure of data and to insert, update, delete and query it.

1.2 About the Project

The Movie Ticket Booking System is a database project that manages the complete flow of booking a cinema ticket: the list of movies, the theatres and screens where they run, the show timings, the customers, the seats they reserve and the payment they make. Today most cinemas sell tickets through websites and mobile applications, and behind every such application there is a well-designed database that decides which seat is free, who has booked it and how much has been paid. This project builds that database layer.

The system is designed with six related tables: movies, theatres, shows, customers, bookings and payments. Primary keys identify every record uniquely, foreign keys connect the tables, and constraints such as NOT NULL, UNIQUE, CHECK and DEFAULT keep the stored data valid. On top of these tables, SQL queries are written to display show schedules, booking details, revenue reports, and a view and transactions are used to cancel a booking safely.

1.3 Why this system is needed

Manual ticket counters cannot show live seat availability, are prone to double booking, and make it difficult to prepare reports. A database-driven system gives accurate seat counts, keeps a permanent record of every booking, supports refunds and cancellations, and gives the management quick answers such as which movie earns the highest revenue or which payment mode is used most.

2. Objectives / Problem Statement

2.1 Problem Statement

Cinemas that handle ticket sales on paper or in spreadsheets face repeated problems: the same seat can be sold twice, the number of available seats is not updated immediately, customer and payment records are difficult to trace, and preparing a revenue report takes a lot of time. The problem is to design and implement a relational database that stores movies, theatres, shows, customers, bookings and payments in a consistent form, and that can answer day-to-day questions of customers and management using SQL.

2.2 Objectives

- To design a normalised relational database for a movie ticket booking system.
- To identify the entities, attributes and relationships and represent them in an ER diagram.
- To create tables in MySQL with primary keys, foreign keys and integrity constraints.
- To insert realistic sample data for movies, theatres, shows, customers, bookings and payments.
- To retrieve information using SELECT queries, JOINs, GROUP BY, HAVING and aggregate functions.
- To create a view that gives a one-line summary of every booking.
- To use transactions (START TRANSACTION, COMMIT) so that cancelling a ticket updates the booking, payment and seat count together.
- To document the project with code and output, and to upload it to GitHub.

2.3 Scope

The project covers the database part of the system: design, creation, population and querying. A front-end (website or mobile app) is not included, but the tables and queries are written so that any front-end can be connected to them later.

3. Modules, Tools and Technologies Used

3.1 Modules

Module	Function	Table(s)
Movie Management	Stores movie title, genre, language, duration, rating and release date.	movies
Theatre Management	Stores theatre name, city and number of screens.	theatres
Show Management	Schedules a movie in a theatre with date, time, ticket price, capacity and available seats.	shows
Customer Management	Stores customer name, unique e-mail and phone number.	customers
Booking Management	Records seats booked by a customer for a show, amount and booking status.	bookings
Payment Management	Records payment mode, amount and payment status for every booking.	payments
Reports	Revenue per movie, repeat customers, booking summary view.	queries, view

3.2 Tools and Technologies

Tool / Technology	Use in this project
MySQL 8.0 (RDBMS)	Database server used to create and manage the database.
MySQL Command Line Client	Used to run the SQL scripts and to capture the output screenshots.
SQL (DDL, DML, DQL, TCL)	CREATE, INSERT, UPDATE, DELETE, SELECT, CREATE VIEW, START TRANSACTION, COMMIT.
ER Modelling	Entity-Relationship diagram to plan tables, keys and relationships.
Git and GitHub	Version control and online submission of the project.
Microsoft Word	Preparation of this project report.
Operating System	Windows 10 / 11 (any OS that supports MySQL).
Hardware	Computer with minimum 4 GB RAM and 1 GB free disk space.

4. Database Design (ER Diagram)

The database movie_booking_db contains six tables. The ER diagram below shows each table with its attributes; PK marks a primary key and FK marks a foreign key. The labels 1, N show the cardinality of each relationship.

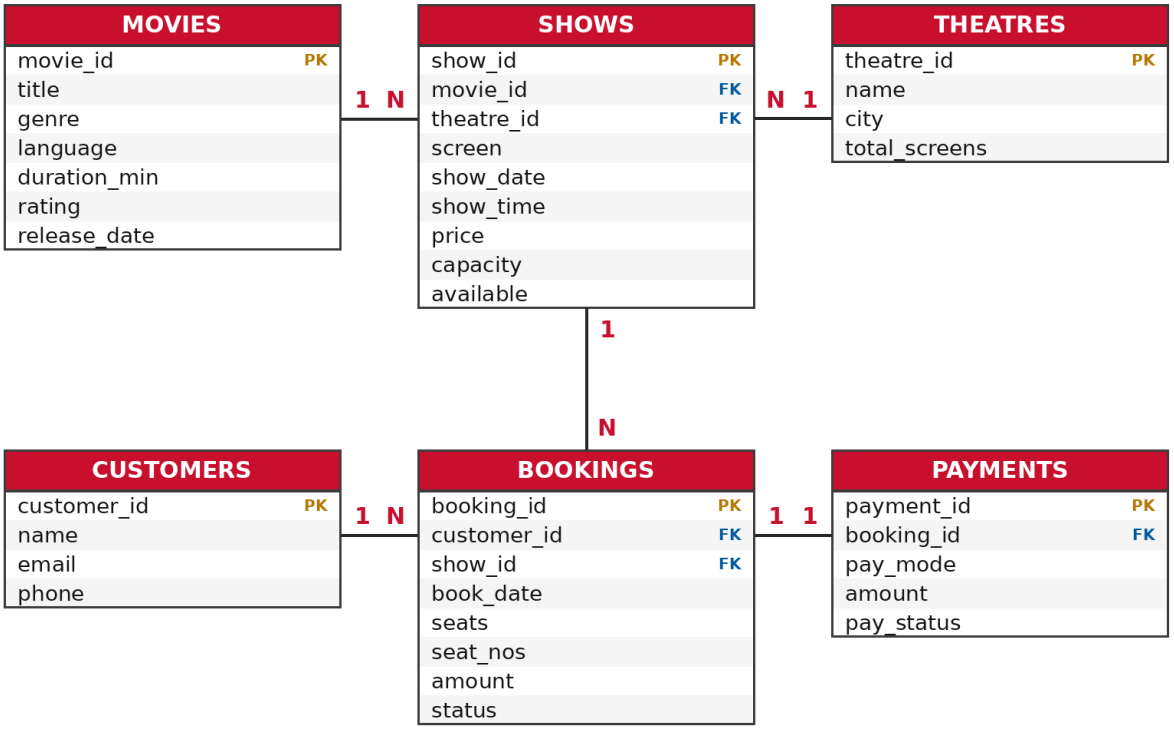


Fig. 4.1: ER diagram of the Movie Ticket Booking System

Relationships

- One movie can be screened in many shows (movies 1 : N shows).
- One theatre can host many shows (theatres 1 : N shows).
- One show can have many bookings (shows 1 : N bookings).
- One customer can make many bookings (customers 1 : N bookings).
- Each booking has one payment record (bookings 1 : 1 payments).

5. Code with Output

5.1 Database and Master Tables

The database is created first, followed by the master tables movies, theatres and customers. Every table has a primary key, and the e-mail of a customer is UNIQUE.

Code: 01_schema_master.sql

```
CREATE DATABASE movie_booking_db;
USE movie_booking_db;

CREATE TABLE movies (
    movie_id      INT PRIMARY KEY,
    title         VARCHAR(100) NOT NULL,
    genre         VARCHAR(30)  NOT NULL,
    language      VARCHAR(20)  NOT NULL,
    duration_min  INT          NOT NULL,
    rating        DECIMAL(2,1),
    release_date  DATE
);

CREATE TABLE theatres (
    theatre_id     INT PRIMARY KEY,
    name           VARCHAR(60) NOT NULL,
    city           VARCHAR(30) NOT NULL,
    total_screens  INT NOT NULL
);

CREATE TABLE customers (
    customer_id  INT PRIMARY KEY,
    name         VARCHAR(60) NOT NULL,
    email        VARCHAR(80) NOT NULL UNIQUE,
    phone        VARCHAR(15) NOT NULL
);
```

Output:

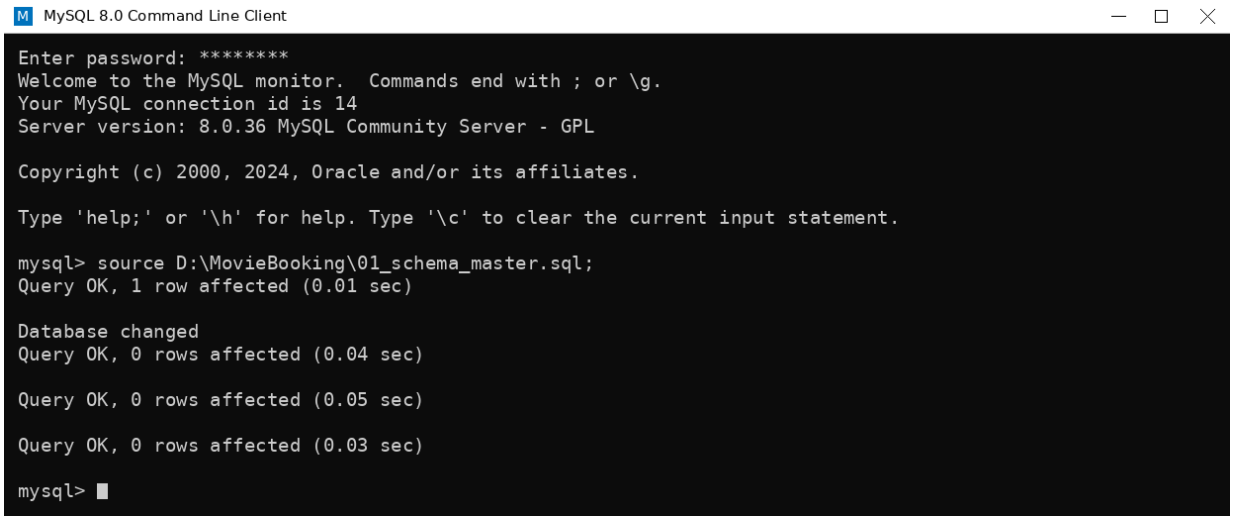


Fig. 5.1: Creating the database and master tables

5.2 Transaction Tables

The tables shows, bookings and payments are linked to the master tables with FOREIGN KEY constraints. CHECK constraints allow only valid seat counts and booking status values.

Code: 02_schema_transaction.sql

```
CREATE TABLE shows (
    show_id     INT PRIMARY KEY,
    movie_id    INT NOT NULL,
    theatre_id  INT NOT NULL,
    screen      INT NOT NULL,
    show_date   DATE NOT NULL,
    show_time   TIME NOT NULL,
    price       DECIMAL(8,2) NOT NULL,
    capacity    INT NOT NULL,
    available   INT NOT NULL,
    FOREIGN KEY (movie_id)   REFERENCES movies(movie_id),
    FOREIGN KEY (theatre_id) REFERENCES theatres(theatre_id)
);

CREATE TABLE bookings (
    booking_id   INT PRIMARY KEY,
    customer_id  INT NOT NULL,
    show_id      INT NOT NULL,
    book_date    DATE NOT NULL,
    seats        INT NOT NULL CHECK (seats > 0),
    seat_nos     VARCHAR(40) NOT NULL,
    amount       DECIMAL(8,2) NOT NULL,
    status       VARCHAR(10) NOT NULL DEFAULT 'Pending'
                 CHECK (status IN ('Confirmed','Pending','Cancelled')),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
    FOREIGN KEY (show_id)     REFERENCES shows(show_id)
);

CREATE TABLE payments (
    payment_id  INT PRIMARY KEY,
    booking_id  INT NOT NULL,
    pay_mode    VARCHAR(20) NOT NULL,
    amount      DECIMAL(8,2) NOT NULL,
    pay_status  VARCHAR(10) NOT NULL,
    FOREIGN KEY (booking_id) REFERENCES bookings(booking_id)
);
```

Output:



Fig. 5.2: Creating the transaction tables and listing all tables with SHOW TABLES

5.3 Inserting Data

Sample data is inserted first into the master tables (six movies, four theatres and six customers).

Code: 03_insert_master.sql

```
INSERT INTO movies VALUES
(1, 'Dangal', 'Sports Drama', 'Hindi', 161, 8.3, '2016-12-23'),
(2, '3 Idiots', 'Comedy Drama', 'Hindi', 170, 8.4, '2009-12-25'),
(3, 'Jawan', 'Action', 'Hindi', 169, 7.0, '2023-09-07'),
(4, 'Oppenheimer', 'Biography', 'English', 180, 8.3, '2023-07-21'),
(5, 'Animal', 'Crime Drama', 'Hindi', 201, 6.3, '2023-12-01'),
(6, 'Carry On Jatta 3', 'Comedy', 'Punjabi', 147, 6.6, '2023-06-29');

INSERT INTO theatres VALUES
(1, 'CineMax Plaza', 'Chandigarh', 5),
(2, 'Star Multiplex', 'Mohali', 4),
(3, 'Silver Screen', 'Ludhiana', 3),
(4, 'Galaxy Cinemas', 'Panchkula', 4);

INSERT INTO customers VALUES
(101, 'Aarav Sharma', 'aarav.sharma@example.com', '9876500001'),
(102, 'Simran Kaur', 'simran.kaur@example.com', '9876500002'),
(103, 'Rohan Verma', 'rohan.verma@example.com', '9876500003'),
(104, 'Priya Singh', 'priya.singh@example.com', '9876500004'),
(105, 'Karan Mehta', 'karan.mehta@example.com', '9876500005'),
(106, 'Harpreet Gill', 'harpreet.gill@example.com', '9876500006');
```

Output:



Fig. 5.3: Inserting movies, theatres and customers

Explanation: each INSERT statement adds several rows at once. MySQL reports the number of rows affected and confirms that there were no duplicate keys or warnings.

Inserting Data (continued)

Next, the transaction data is inserted: eight shows, eight bookings and eight payments. The available seats of every show are already reduced by the seats booked.

Code: 04_insert_transaction.sql

```
INSERT INTO shows VALUES
(1001, 1, 1, 1, '2026-10-22', '10:00:00', 200.00, 120, 118),
(1002, 2, 1, 2, '2026-10-22', '13:30:00', 180.00, 100, 99),
(1003, 3, 2, 1, '2026-10-22', '18:00:00', 250.00, 150, 146),
(1004, 4, 2, 3, '2026-10-23', '19:30:00', 300.00, 90, 87),
(1005, 5, 3, 2, '2026-10-23', '21:00:00', 220.00, 110, 105),
(1006, 6, 4, 1, '2026-10-24', '16:00:00', 160.00, 80, 78),
(1007, 3, 1, 3, '2026-10-24', '20:15:00', 280.00, 140, 138),
(1008, 1, 3, 1, '2026-10-25', '11:00:00', 150.00, 100, 97);

INSERT INTO bookings VALUES
(5001, 101, 1001, '2026-10-18', 2, 'A1,A2', 400.00, 'Confirmed'),
(5002, 102, 1003, '2026-10-18', 4, 'C5,C6,C7,C8', 1000.00, 'Confirmed'),
(5003, 103, 1004, '2026-10-19', 3, 'B3,B4,B5', 900.00, 'Confirmed'),
(5004, 101, 1007, '2026-10-19', 2, 'D9,D10', 560.00, 'Confirmed'),
(5005, 104, 1002, '2026-10-19', 1, 'F12', 180.00, 'Confirmed'),
(5006, 105, 1005, '2026-10-20', 5, 'E1,E2,E3,E4,E5', 1100.00, 'Confirmed'),
(5007, 106, 1006, '2026-10-20', 2, 'G7,G8', 320.00, 'Confirmed'),
(5008, 102, 1008, '2026-10-20', 3, 'H4,H5,H6', 450.00, 'Pending');

INSERT INTO payments VALUES
(9001, 5001, 'UPI', 400.00, 'Success'),
(9002, 5002, 'Credit Card', 1000.00, 'Success'),
(9003, 5003, 'Debit Card', 900.00, 'Success'),
(9004, 5004, 'UPI', 560.00, 'Success'),
(9005, 5005, 'Cash', 180.00, 'Success'),
(9006, 5006, 'Net Banking', 1100.00, 'Success'),
(9007, 5007, 'UPI', 320.00, 'Success'),
(9008, 5008, 'UPI', 450.00, 'Pending');
```

Output:



Fig. 5.4: Inserting shows, bookings and payments

5.4 Viewing Table Data

The stored data is displayed with SELECT * statements.

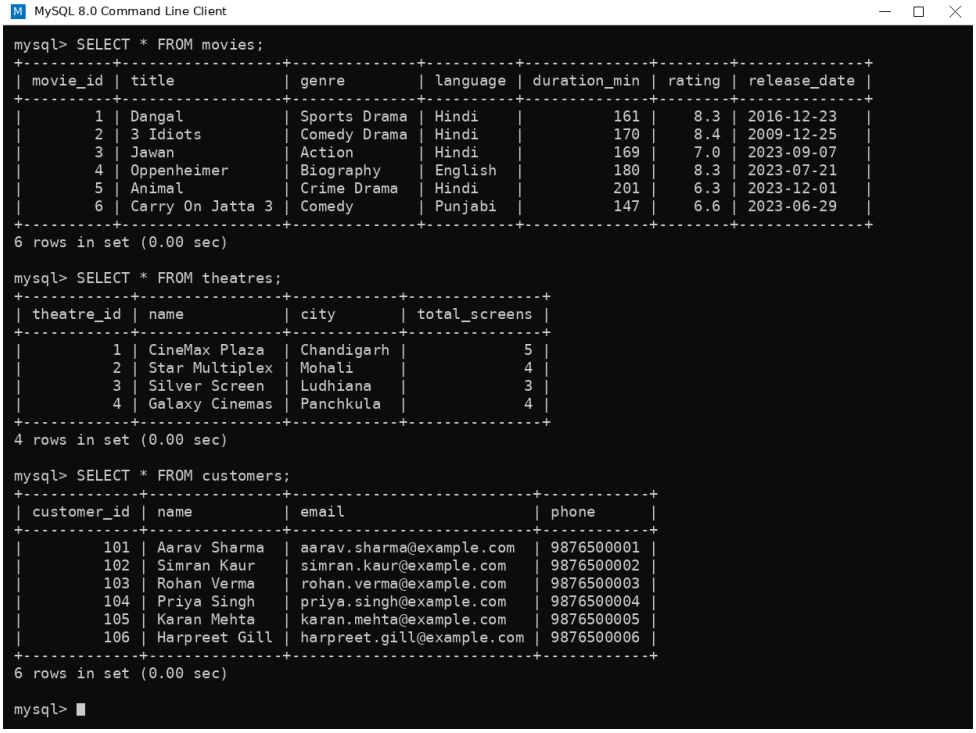


Fig. 5.5: Contents of movies, theatres and customers

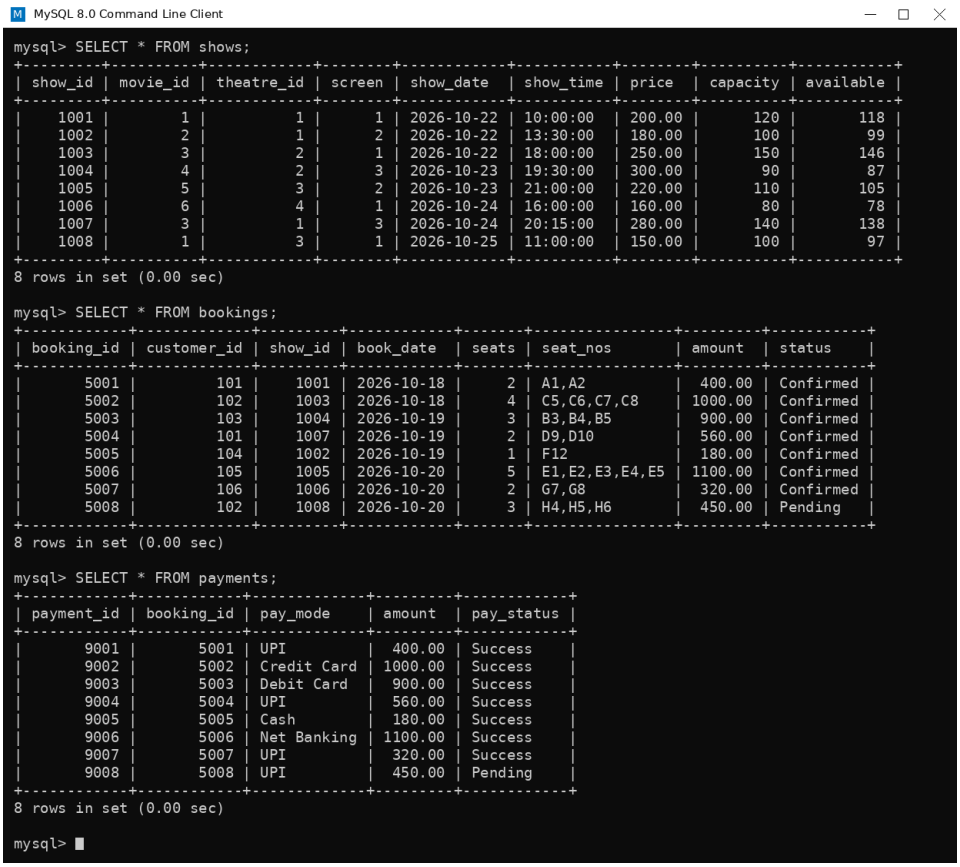


Fig. 5.6: Contents of shows, bookings and payments

5.5 SQL Queries

Query 1: Show schedule with movie and theatre name (JOIN of three tables)

```
SELECT s.show_id, m.title, t.name AS theatre, s.show_date, s.show_time, s.price
FROM shows s
JOIN movies m   ON s.movie_id = m.movie_id
JOIN theatres t ON s.theatre_id = t.theatre_id
ORDER BY s.show_date, s.show_time;
```

MySQL 8.0 Command Line Client

```
mysql> SELECT s.show_id, m.title, t.name AS theatre, s.show_date, s.show_time, s.price
    -> FROM shows s
    -> JOIN movies m   ON s.movie_id = m.movie_id
    -> JOIN theatres t ON s.theatre_id = t.theatre_id
    -> ORDER BY s.show_date, s.show_time;
+---------+-----------------+---------------+------------+-----------+--------+
| show_id | title           | theatre       | show_date  | show_time | price  |
+---------+-----------------+---------------+------------+-----------+--------+
|    1001 | Dangal          | CineMax Plaza | 2026-10-22 | 10:00:00  | 200.00 |
|    1002 | 3 Idiots        | CineMax Plaza | 2026-10-22 | 13:30:00  | 180.00 |
|    1003 | Jawan           | Star Multiplex| 2026-10-22 | 18:00:00  | 250.00 |
|    1004 | Oppenheimer     | Star Multiplex| 2026-10-23 | 19:30:00  | 300.00 |
|    1005 | Animal          | Silver Screen | 2026-10-23 | 21:00:00  | 220.00 |
|    1006 | Carry On Jatta 3| Galaxy Cinemas| 2026-10-24 | 16:00:00  | 160.00 |
|    1007 | Jawan           | CineMax Plaza | 2026-10-24 | 20:15:00  | 280.00 |
|    1008 | Dangal          | Silver Screen | 2026-10-25 | 11:00:00  | 150.00 |
+---------+-----------------+---------------+------------+-----------+--------+
8 rows in set (0.00 sec)

mysql>
```

Fig. 5.7: Output of Query 1

Query 2: Booking details with customer name and movie title (JOIN of four tables)

```
SELECT b.booking_id, c.name AS customer, m.title AS movie, s.show_date,
       b.seats, b.amount, b.status
FROM bookings b
JOIN customers c ON b.customer_id = c.customer_id
JOIN shows s     ON b.show_id = s.show_id
JOIN movies m    ON s.movie_id = m.movie_id
ORDER BY b.booking_id;
```

MySQL 8.0 Command Line Client

```
mysql> SELECT b.booking_id, c.name AS customer, m.title AS movie, s.show_date,
    ->        b.seats, b.amount, b.status
    -> FROM bookings b
    -> JOIN customers c ON b.customer_id = c.customer_id
    -> JOIN shows s     ON b.show_id = s.show_id
    -> JOIN movies m    ON s.movie_id = m.movie_id
    -> ORDER BY b.booking_id;
+------------+--------------+-----------------+------------+-------+---------+-----------+
| booking_id | customer     | movie           | show_date  | seats | amount  | status    |
+------------+--------------+-----------------+------------+-------+---------+-----------+
|       5001 | Aarav Sharma | Dangal          | 2026-10-22 |     2 |  400.00 | Confirmed |
|       5002 | Simran Kaur  | Jawan           | 2026-10-22 |     4 | 1000.00 | Confirmed |
|       5003 | Rohan Verma  | Oppenheimer     | 2026-10-23 |     3 |  900.00 | Confirmed |
|       5004 | Aarav Sharma | Jawan           | 2026-10-24 |     2 |  560.00 | Confirmed |
|       5005 | Priya Singh  | 3 Idiots        | 2026-10-22 |     1 |  180.00 | Confirmed |
|       5006 | Karan Mehta  | Animal          | 2026-10-23 |     5 | 1100.00 | Confirmed |
|       5007 | Harpreet Gill| Carry On Jatta 3| 2026-10-24 |     2 |  320.00 | Confirmed |
|       5008 | Simran Kaur  | Dangal          | 2026-10-25 |     3 |  450.00 | Pending   |
+------------+--------------+-----------------+------------+-------+---------+-----------+
8 rows in set (0.00 sec)

mysql>
```

Fig. 5.8: Output of Query 2

SQL Queries (continued)

Query 3: Revenue per movie from confirmed bookings (GROUP BY, SUM, COUNT)

```
SELECT m.title, COUNT(b.booking_id) AS total_bookings,
       SUM(b.seats) AS tickets_sold, SUM(b.amount) AS revenue
FROM movies m
JOIN shows s    ON m.movie_id = s.movie_id
JOIN bookings b ON s.show_id = b.show_id
WHERE b.status = 'Confirmed'
GROUP BY m.title
ORDER BY revenue DESC;
```

MySQL 8.0 Command Line Client

```
mysql> SELECT m.title, COUNT(b.booking_id) AS total_bookings,
    ->        SUM(b.seats) AS tickets_sold, SUM(b.amount) AS revenue
    -> FROM movies m
    -> JOIN shows s    ON m.movie_id = s.movie_id
    -> JOIN bookings b ON s.show_id = b.show_id
    -> WHERE b.status = 'Confirmed'
    -> GROUP BY m.title
    -> ORDER BY revenue DESC;
+-----------------+----------------+--------------+---------+
| title           | total_bookings | tickets_sold | revenue |
+-----------------+----------------+--------------+---------+
| Jawan           |              2 |            6 | 1560.00 |
| Animal          |              1 |            5 | 1100.00 |
| Oppenheimer     |              1 |            3 |  900.00 |
| Dangal          |              1 |            2 |  400.00 |
| Carry On Jatta 3 |              1 |            2 |  320.00 |
| 3 Idiots        |              1 |            1 |  180.00 |
+-----------------+----------------+--------------+---------+
6 rows in set (0.00 sec)

mysql>
```

Fig. 5.9: Output of Query 3

Query 4: Customers with more than one booking (HAVING)

```
SELECT c.name, COUNT(*) AS total_bookings, SUM(b.amount) AS total_spent
FROM customers c
JOIN bookings b ON c.customer_id = b.customer_id
GROUP BY c.customer_id, c.name
HAVING COUNT(*) > 1;
```

MySQL 8.0 Command Line Client

```
mysql> SELECT c.name, COUNT(*) AS total_bookings, SUM(b.amount) AS total_spent
    -> FROM customers c
    -> JOIN bookings b ON c.customer_id = b.customer_id
    -> GROUP BY c.customer_id, c.name
    -> HAVING COUNT(*) > 1;
+-------------+----------------+-------------+
| name        | total_bookings | total_spent |
+-------------+----------------+-------------+
| Aarav Sharma |              2 |      960.00 |
| Simran Kaur  |              2 |     1450.00 |
+-------------+----------------+-------------+
2 rows in set (0.00 sec)

mysql>
```

Fig. 5.10: Output of Query 4

5.6 View, Transaction and Delete

A view gives a one-line summary of every booking. A transaction cancels booking 5006: the booking status, the payment status and the seat count are changed together, and COMMIT makes the change permanent. An unpaid pending booking is then deleted.

Code: 06_view_and_transactions.sql

```
-- View: one-line summary of every booking
CREATE VIEW booking_summary AS
SELECT b.booking_id, c.name AS customer, m.title AS movie,
       t.name AS theatre, s.show_date, s.show_time, b.seats, b.status
FROM bookings b
JOIN customers c ON b.customer_id = c.customer_id
JOIN shows s     ON b.show_id = s.show_id
JOIN movies m    ON s.movie_id = m.movie_id
JOIN theatres t  ON s.theatre_id = t.theatre_id;

SELECT * FROM booking_summary WHERE show_date = '2026-10-23';

-- Transaction: cancel booking 5006 and release its seats
START TRANSACTION;
UPDATE bookings SET status = 'Cancelled' WHERE booking_id = 5006;
UPDATE payments SET pay_status = 'Refunded' WHERE booking_id = 5006;
UPDATE shows    SET available = available + 5 WHERE show_id = 1005;
COMMIT;

SELECT show_id, capacity, available FROM shows WHERE show_id = 1005;

-- Delete an unpaid (pending) booking and release its seats
DELETE FROM payments WHERE booking_id = 5008;
DELETE FROM bookings WHERE booking_id = 5008;
UPDATE shows SET available = available + 3 WHERE show_id = 1008;
```

Output:

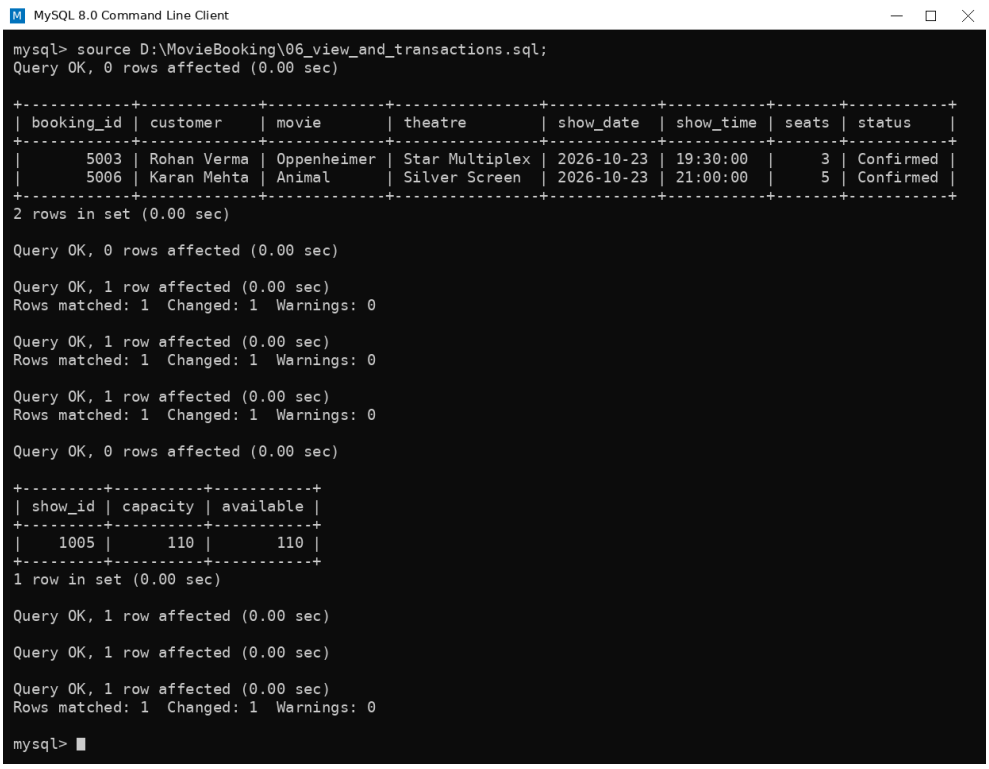


Fig. 5.12: Output of the view, transaction and delete statements

6. Conclusion, Future Scope and References

6.1 Conclusion

The Movie Ticket Booking System shows how a relational database can manage the whole booking process of a cinema. Six tables connected through primary and foreign keys store movies, theatres, shows, customers, bookings and payments without duplication. Constraints protect the data from invalid values, JOIN queries combine information from several tables, aggregate queries give revenue and payment reports, the view simplifies reporting, and the transaction guarantees that a cancellation updates every related table together. Through this project I practised database design, ER modelling and the main groups of SQL commands (DDL, DML, DQL and TCL).

6.2 Future Scope

- Add a seats table so that every seat of every screen is tracked individually and double booking is blocked by the database.
- Add triggers to update the available seats automatically after every booking or cancellation.
- Connect the database to a website or mobile application using Python, Java or PHP.
- Add user login, discount coupons and online payment gateway records.
- Create stored procedures and dashboards for daily and monthly sales reports.

6.3 GitHub Repository

The complete project (SQL scripts, ER diagram and screenshots) is available at:
https://github.com/<your-username>/movie-ticket-booking-system

6.4 References

- MySQL 8.0 Reference Manual, Oracle Corporation, https://dev.mysql.com/doc/
- R. Elmasri and S. Navathe, Fundamentals of Database Systems, Pearson.
- A. Silberschatz, H. Korth and S. Sudarshan, Database System Concepts, McGraw-Hill.
- W3Schools SQL Tutorial, https://www.w3schools.com/sql/